Advanced SQL Assignment

Q1) You are working on a data warehouse for a retail company. The company wants to be able to quickly report on sales by product category, region, and time period. The current database schema is normalized, but this is resulting in slow performance for these reports. How would you denormalize the schema to improve performance?

Answer:
To improve reporting performance, the normalized schema can be denormalized using a Star Schema.

A star schema consists of:

1. **Fact table:** Stores measurable business data, such as sales amount, quantity, and transaction count.
2. **Dimension tables:** Store descriptive information used for analysis, such as product, customer, region, and date.

Proposed schema:

1. Fact_Sales
   - sale_id
   - product_id
   - region_id
   - date_id
   - quantity_sold
   - sales_amount
2. Dim_Product
   - product_id
   - product_name
   - category
3. Dim_Region
   - region_id
   - region_name
   - country

4. Dim_Date
    - date_id
    - date
    - month
    - quarter
    - year

How denormalization improves performance:

1. Reduces the number of joins required for reporting.
2. Stores frequently used descriptive attributes in dimension tables.
3. Makes aggregation and grouping by category, region, and time easier.
4. Improves query performance for analytical workloads.

Example query:

```
SELECT p.category,r.region_name,d.year,SUM(f.sales_amount) AS total_sales
FROM Fact_Sales f
JOIN Dim_Product p ON f.product_id = p.product_id
JOIN Dim_Region r ON f.region_id = r.region_id
JOIN Dim_Date d ON f.date_id = d.date_id
GROUP BY p.category, r.region_name, d.year;
```

Conclusion:
A star schema improves analytical query performance by reducing join complexity and organizing data around business measures and dimensions. The trade-off is some data redundancy and additional effort to maintain consistency during data updates.

Q2) Your company has a data warehouse that contains a table of customer orders. You want to create a view that shows the total order value for each customer. How would you create this view?

Answer:
A View is a virtual table based on the result of an SQL query. It stores the query definition rather than storing the actual result data.

To display the total order value for each customer, create a view using `CREATE VIEW`, `JOIN`, `GROUP BY`, and `SUM()`.

SQL query:

```sql
CREATE VIEW customer_order_totals AS
SELECT
    customer_id,
    SUM(total_amount) AS total_order_value
FROM Orders
GROUP BY customer_id;
```

To retrieve the data:

```sql
SELECT *
FROM customer_order_totals;
```

Explanation:

1. `CREATE VIEW`: Creates a virtual table.
2. `SUM(total_amount)`: Calculates the total order value.
3. `GROUP BY customer_id`: Groups orders belonging to each customer.
4. The view can be queried like a regular table.

Advantages:

1. Simplifies frequently used queries.
2. Avoids rewriting aggregation logic.
3. Provides controlled access to selected data.
4. Reflects changes in the underlying tables when queried.

Conclusion

A view provides a reusable and simplified way to access customer-wise order summaries without storing a separate copy of the aggregated data.

Q3) Your company has a data warehouse that contains a table of employee data. You want to create a procedure that calculates the employee salary history for each employee. How would you create this procedure?

Answer:
A stored procedure can be created to retrieve the salary history of employees by using an employee table that stores salary changes over time.

Assume the table contains `employee_id`, `salary`, and `effective_date` columns:

```
CREATE OR REPLACE PROCEDURE calculate_salary_history()
LANGUAGE SQL
AS $$
    SELECT employee_id,salary,effective_date,
        LAG(salary) OVER (
            PARTITION BY employee_id
            ORDER BY effective_date
        ) AS previous_salary
    FROM employee
    ORDER BY employee_id, effective_date;
$$;
```

Execute the procedure:

```
CALL calculate_salary_history();
```

Explanation:

1. `PARTITION BY employee_id` processes each employee's salary records separately.
2. `ORDER BY effective_date` arranges salary records chronologically.
3. `LAG(salary)` retrieves the employee's salary from the previous record.
4. The first salary record for each employee has `NULL` as its previous salary.

> **Note**
>
> This procedure assumes the salary changes are already stored as separate rows in the `employee` table. In PostgreSQL, a procedure does not directly return a result set to the caller using a plain `SELECT` like this. For a reusable query that returns these rows, a function is more appropriate.

Q4) Your company has a data warehouse that contains a table of customer data. You want to track changes to the customer data over time. How would you implement a Slowly Changing Dimension (SCD) type 2 for this table?

Answer:
SCD Type 2 maintains the complete history of changes to customer data. Instead of overwriting an existing record, it expires the old record and inserts a new record whenever customer information changes.

1. Create the customer dimension table

Include columns to track the validity period and identify the current record.

```
CREATE TABLE customer_dimension (
    customer_id INT,
    customer_name VARCHAR(100),
    city VARCHAR(50),
    start_date DATE,
    end_date DATE,
    is_current BOOLEAN
);
```

2. Expire the existing record

When customer information changes, update the existing record by setting its `end_date` and marking it as no longer current.

```
UPDATE customer_dimension
SET
    end_date = CURRENT_DATE,
    is_current = FALSE
WHERE customer_id = 101 AND is_current = TRUE;
```

3. Insert the updated customer record

Insert the new customer information as a new version.

```
INSERT INTO customer_dimension
VALUES (
    101,
    'Jhon Smith',
    'Mumbai',
    CURRENT_DATE,
    NULL,
    TRUE
);
```

4. Retrieve the current customer records

```
SELECT *
FROM customer_dimension
WHERE is_current = TRUE;
```

Explanation:

1. Start Date: Date from which the customer record becomes valid.
2. End Date: Date until which the record remains valid. `NULL` indicates the current record.
3. Is Current: Identifies whether the record is the latest version.
4. Old records: Retained for historical analysis instead of being deleted.

Example:

Customer ID	Customer Name	City	Start Date	End Date	Is Current
101	John Smith	Bangalore	2024-01-01	2026-11-09	FALSE
101	John Smith	Mumbai	2026-11-09	NULL	TRUE

Conclusion

SCD Type 2 preserves historical customer information by maintaining multiple versions of each customer record. This allows the data warehouse to track changes and analyse customer information as it existed at different points in time.

The update and insert statements should be executed within a single transaction to ensure both operations succeed or fail together.

Q5) Scenario: You are working on a data warehouse for a social media company. The company wants to be able to track the number of active users on a daily basis. The current database schema is normalized, but this is resulting in slow performance for this report. Question: How would you denormalize the schema to improve performance, while still maintaining data integrity?

Answer:

To improve report performance, a pre-aggregated summary table can be created to store the daily active user count instead of calculating it from multiple normalized tables every time.

1. Create a daily active users table

```sql
CREATE TABLE daily_active_users (
    activity_date DATE PRIMARY KEY,
    active_user_count INT NOT NULL
);
```

2. Populate the summary table

Assume an `activity` table stores user activity with `user_id` and `activity_date`:

```sql
INSERT INTO daily_active_users (activity_date, active_user_count)
SELECT activity_date,COUNT(DISTINCT user_id) AS active_user_count
FROM activity
GROUP BY activity_date;
```

`COUNT(DISTINCT user_id)` ensures that each user is counted only once per day, even if they perform multiple activities.

3. Retrieve daily active users

```sql
SELECT activity_date, active_user_count
FROM daily_active_users
ORDER BY activity_date;
```

4. Maintain data integrity

1. Use `activity_date` as the primary key to prevent duplicate daily summary records.
2. Use `NOT NULL` constraints to ensure valid summary values.
3. Refresh the summary table whenever new activity data is loaded.
4. Perform updates and inserts within transactions to keep summary data consistent with the source tables.
5. Use scheduled ETL jobs to periodically recalculate or update daily counts.

Reason for this design:

Normalized schema	Denormalized summary table
Calculates counts from detailed activity records.	Stores pre-calculated daily counts.
Requires aggregation for every report.	Retrieves counts directly.
Suitable for storing detailed activity.	Suitable for faster analytical reporting.

Conclusion

A pre-aggregated daily summary table improves reporting performance by reducing repeated calculations. The original normalized tables remain the source of truth, while the summary table provides faster analytical access. SCD Type 2 is not necessary here because the requirement is to track daily active user counts, not historical changes to customer attributes.